

Section A: Vector Databases

A1. Explain in your own words what a vector database is and how it's different from a traditional database like MySQL or PostgreSQL.

- Vector databases store data as vectors. Vectors mean a long list of numbers representing meaning.
- Vector database is built for to search similarity between two points
- Vector databases include indexing using which we find similarity between two points

A2. Take any sentence of your choice. Describe what happens to it step by step from raw text to being stored in a vector database. Name each stage.

Sentence = "Hello this is Tejal"

Step1 – Token Generation(subword method)

- Split into tokens = ['hello','this','is','tej','al']

Step2- Embeddings

- Tokens here passed to model which outputs vectors
- [0.2,0.1,0.8,0.6,0.1]

Step3 – Indexing

- The vector is stored in databases and added indexing like hnsw

Step4 – Stored in VDB

A3. Why can't we just use a normal database index (like a B-tree) to search through embeddings? Explain the core difference between exact lookup and similarity search.

- B-trees work great for exact ranges like "find all ages between 20-30" because data can be sorted and compared directly
- But embeddings are high-dimensional vectors so there's no natural "sort order" for a number vector, so you can't binary-search through

them. Instead, you need similarity search like cosine similarity or Euclidean distance to compare how "close" two vectors are

A4. You're given two vectors: $A = [1, 2, 3]$ $B = [2, 4, 6]$ Calculate the cosine similarity between A and B by hand, showing your steps. What does the result tell you about these two vectors?

The image shows a handwritten solution for calculating the cosine similarity between two vectors, A and B. The vectors are defined as $A = [1, 2, 3]$ and $B = [2, 4, 6]$. The formula for cosine similarity is given as $\cos \theta = \frac{A \cdot B}{|A| \times |B|}$. The calculation proceeds in three steps: 1. Dot product: $A \cdot B = (1 \times 2) + (2 \times 4) + (3 \times 6) = 2 + 8 + 18 = 28$. 2. Magnitude of A: $|A| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{1 + 4 + 9} = \sqrt{14} \approx 3.742$. 3. Magnitude of B: $|B| = \sqrt{2^2 + 4^2 + 6^2} = \sqrt{4 + 16 + 36} = \sqrt{56} \approx 7.483$. Finally, the cosine similarity is calculated as $\frac{A \cdot B}{|A| |B|} = \frac{28}{3.742 \times 7.483} \approx \frac{28}{28} \approx 1$.

$$A = [1, 2, 3] \quad B = [2, 4, 6]$$

Formula $\cos \theta = \frac{A \cdot B}{|A| \times |B|}$

Step ① dot product

$$A \cdot B = (1 \times 2) + (2 \times 4) + (3 \times 6) = 2 + 8 + 18 = 28$$

Step ② magnitude

$$|A| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{1 + 4 + 9} = \sqrt{14} = 3.742$$
$$|B| = \sqrt{2^2 + 4^2 + 6^2} = \sqrt{4 + 16 + 36} = \sqrt{56} = 7.483$$

Step ③ All in

$$\frac{A \cdot B}{|A| |B|} = \frac{28}{3.742 \times 7.483} \approx \frac{28}{28} \approx 1$$

A5. Explain why a vector database is essential for RAG specifically. What would break if you removed it from the pipeline?

- RAG basically works by finding chunks that are close in *meaning* not just matching exact words.
- If you remove the vector database, you lose the fast way to find "which chunks actually relate to this question." you'd be stuck with plain keyword search,
- so basically, no vector db = no smart retrieval = the whole RAG thing just stops working properly

Section C : Advanced Rag

C1. You're building a RAG system for a company's product manuals. These manuals have text instructions along with diagrams and photos showing how to assemble the product. A user asks "how do I attach the side panel," and the answer is shown entirely in a diagram, with no accompanying text. Explain how you would approach this with multimodal RAG. What would go wrong if you used a text-only RAG pipeline here?

- If the answer to "how do I attach the side panel" only exists in a diagram (no text), a text-only RAG pipeline would completely fail it has nothing to retrieve, since there's no text chunk describing that step. To fix this, I'd use multimodal RAG: extract images/diagrams from the manual will generate embeddings for them using a multimodal model and store those in the vector DB
- When a user asks a question the query gets embedded and compared against both text *and* image embeddings so it can retrieve the relevant diagram even when no text matches. The final answer could then combine the image with a explanation, or just show the diagram directly.

C2. Briefly compare the two approaches to multimodal retrieval covered in lecture: unified embedding space vs separate indexes with fusion. Which one would you pick for the product manual scenario above, and why?

- **Unified embedding space:** text and images are embedded into the *same* vector space so you can directly compare a text query against an image using one similarity search
- **Separate indexes with fusion:** text and images are embedded separately then results from both are combined ("fused") afterward using some method.
- **For the product manual case, I'd pick unified embedding space.** Since the question could be answered by *either* text or an image, having one shared space, so we don't have to merge text and image
-